Machine Learning: A Comprehensive Textbook

Solutions to Chapter 6 Exercises

Ensemble Methods: Bagging, Boosting, and Stacking

Solutions Manual

Exercise 6.1 — Bias-Variance Decomposition

Prove $\mathbb{E}[(y-\hat{f})^2] = \text{Bias}^2 + \text{Var} + \sigma^2$ from scratch.

Solution. Let $y = f(x) + \epsilon$ with $\mathbb{E}[\epsilon] = 0$, $\text{Var}(\epsilon) = \sigma^2$, ϵ independent of the training data (hence of $\hat{f}$). Let $\hat{f} = \hat{f}(x)$ denote the model's prediction at a fixed test point x (a random variable due to randomness in the training set). We compute the expected squared error over both the noise ϵ and the training-set randomness in $\hat{f}$:

$$\mathbb{E}[(y-\hat{f})^2] = \mathbb{E}[(f(x)+\epsilon-\hat{f})^2].$$

Add and subtract $\mathbb{E}[\hat{f}]$:

$$= \mathbb{E}\Big[\big((f(x)-\mathbb{E}[\hat{f}]) + (\mathbb{E}[\hat{f}]-\hat{f}) + \epsilon\big)^2\Big].$$

Expand the square of the sum of three terms. Let $a = f(x) - \mathbb{E}[\hat{f}]$ (a constant, not random), $b = \mathbb{E}[\hat{f}] - \hat{f}$ (random, mean zero: $\mathbb{E}[b] = 0$), $c = \epsilon$ (random, mean zero, independent of b):

$$\mathbb{E}[(a+b+c)^2] = a^2 + \mathbb{E}[b^2] + \mathbb{E}[c^2] + 2a\mathbb{E}[b] + 2a\mathbb{E}[c] + 2\mathbb{E}[bc].$$

- $2a\mathbb{E}[b] = 2a \cdot 0 = 0$ since $\mathbb{E}[b] = \mathbb{E}[\mathbb{E}[\hat{f}] - \hat{f}] = \mathbb{E}[\hat{f}] - \mathbb{E}[\hat{f}] = 0$.
- $2a\mathbb{E}[c] = 2a \cdot 0 = 0$ since $\mathbb{E}[\epsilon] = 0$.
- $2\mathbb{E}[bc] = 2\mathbb{E}[b]\mathbb{E}[c] = 0$ since b (a function of the training set) and $c = \epsilon$ (fresh test noise) are independent, and $\mathbb{E}[c] = 0$.

So all cross terms vanish, leaving

$$\mathbb{E}[(y-\hat{f})^2] = a^2 + \mathbb{E}[b^2] + \mathbb{E}[c^2] = \big(f(x) - \mathbb{E}[\hat{f}]\big)^2 + \mathbb{E}\big[(\hat{f} - \mathbb{E}[\hat{f}])^2\big] + \mathbb{E}[\epsilon^2].$$

Identify each term: $\big(f(x) - \mathbb{E}[\hat{f}]\big)^2 = \text{Bias}(\hat{f})^2$; $\mathbb{E}[(\hat{f} - \mathbb{E}[\hat{f}])^2] = \text{Var}(\hat{f})$; $\mathbb{E}[\epsilon^2] = \text{Var}(\epsilon) = \sigma^2$ (since $\mathbb{E}[\epsilon] = 0$). Hence

$$\mathbb{E}[(y-\hat{f})^2] = \text{Bias}(\hat{f})^2 + \text{Var}(\hat{f}) + \sigma^2. \qquad \blacksquare$$

Exercise 6.2 — Ensemble Variance

Derive $\rho\sigma^2 + (1-\rho)\sigma^2/M$. When does averaging M models achieve zero variance? Is this achievable in practice?

Solution. Let $\hat{f}_1, \ldots, \hat{f}_M$ be M predictors, each with variance $\text{Var}(\hat{f}_m) = \sigma^2$, and pairwise correlation ρ between any two distinct predictors ($\text{Cov}(\hat{f}_m, \hat{f}_{m'}) = \rho\sigma^2$ for $m \neq m'$). The ensemble average is $\bar{f} = \frac{1}{M}\sum_{m=1}^{M} \hat{f}_m$. Its variance:

$$\text{Var}(\bar{f}) = \text{Var}\Big(\frac{1}{M}\sum_m \hat{f}_m\Big) = \frac{1}{M^2}\text{Var}\Big(\sum_m \hat{f}_m\Big) = \frac{1}{M^2}\left[\sum_m \text{Var}(\hat{f}_m) + \sum_{m \neq m'} \text{Cov}(\hat{f}_m, \hat{f}_{m'})\right].$$

There are M variance terms (each $= \sigma^2$) and $M(M-1)$ ordered covariance pairs (each $= \rho\sigma^2$):

$$\text{Var}(\bar{f}) = \frac{1}{M^2}\Big[M\sigma^2 + M(M-1)\rho\sigma^2\Big] = \frac{\sigma^2}{M} + \frac{M(M-1)}{M^2}\rho\sigma^2 = \frac{\sigma^2}{M} + \left(1 - \frac{1}{M}\right)\rho\sigma^2.$$

Rearranging:

$$\text{Var}(\bar{f}) = \rho\sigma^2 + \frac{(1-\rho)\sigma^2}{M}. \qquad \blacksquare$$

Zero variance condition. As $M \to \infty$, the second term $\to 0$, leaving $\text{Var}(\bar{f}) \to \rho\sigma^2$. This equals exactly zero only if $\rho = 0$ (fully uncorrelated base learners) — then $\text{Var}(\bar{f}) \to 0$ as $M \to \infty$. If $\rho > 0$ (any positive correlation among base learners), the ensemble variance has an irreducible floor of $\rho\sigma^2 > 0$ no matter how large M is.

Achievable in practice? Exactly zero correlation ($\rho = 0$) between base learners trained on (even resampled) versions of the *same* underlying dataset is essentially unachievable in practice — bootstrap samples overlap substantially (each bootstrap sample shares roughly 63.2% of its distinct points with the full training set, cf. Exercise 6.3), and all base learners are estimating the same underlying $f(x)$, so their errors are inevitably somewhat correlated. In practice ρ is small but strictly positive for well-designed ensembles (e.g. random forests reduce ρ specifically via feature subsampling, on top of bagging's row-subsampling, precisely to push ρ closer to zero and get closer to this ideal), but the $\rho\sigma^2$ floor is never fully eliminated — this is exactly why methods like Random Forest actively try to *decorrelate* trees (not just bag them) as an explicit design goal.

Exercise 6.3 — Bootstrap Probability

Show that as $n \to \infty$, $P(\text{example absent from bootstrap sample}) \to e^{-1} \approx 0.368$.

Solution. A bootstrap sample of size n is drawn *with replacement* from n training examples, so each of the n draws independently selects any particular example with probability $1/n$. The probability that a *specific* example is *not* selected on a single draw is $1 - \frac{1}{n}$. Since the n draws are independent, the probability that this example is never selected across all n draws is

$$P(\text{absent}) = \left(1 - \frac{1}{n}\right)^n.$$

Take the limit as $n \to \infty$. Recall the standard limit $\lim_{n\to\infty}\left(1 + \frac{x}{n}\right)^n = e^x$; here $x = -1$:

$$\lim_{n\to\infty}\left(1 - \frac{1}{n}\right)^n = e^{-1} \approx 0.3679.$$

(More rigorously: take logs, $n\ln(1-1/n) = n\big(-\frac{1}{n} - \frac{1}{2n^2} - O(n^{-3})\big) = -1 - \frac{1}{2n} - O(n^{-2}) \to -1$ as $n \to \infty$, using the Taylor expansion $\ln(1-x) = -x - x^2/2 - \ldots$ for $x = 1/n \to 0$; exponentiating gives the limit e^{-1}.)

So as $n \to \infty$, $P(\text{a specific example absent from a bootstrap sample}) \to e^{-1} \approx 36.8\%$. $\blacksquare$ Equivalently, each bootstrap sample contains, in expectation, only about $1 - e^{-1} \approx 63.2\%$ of the *distinct* original examples (the remainder are duplicates), and the excluded $\sim 36.8\%$ forms the "out-of-bag" (OOB) set — used for OOB error estimation in bagging/random forests as a built-in, free validation set requiring no separate held-out split.

Exercise 6.4 — AdaBoost Update Derivation

Show $\alpha_t = \frac{1}{2}\ln\frac{1-\epsilon_t}{\epsilon_t}$ minimises $\sum_i D_t(i)\exp(-\alpha_t y^{(i)} h_t(x^{(i)}))$ over α_t.

Solution. Define $Z(\alpha_t) := \sum_{i=1}^n D_t(i)\exp\big(-\alpha_t y^{(i)} h_t(x^{(i)})\big)$, where $y^{(i)}, h_t(x^{(i)}) \in \{-1,+1\}$, so $y^{(i)}h_t(x^{(i)}) = +1$ if h_t correctly classifies example i, and $= -1$ if misclassified. Split the sum into correctly and incorrectly classified examples:

$$Z(\alpha_t) = \sum_{i:\,\text{correct}} D_t(i)e^{-\alpha_t} + \sum_{i:\,\text{incorrect}} D_t(i)e^{\alpha_t} = e^{-\alpha_t}\underbrace{\sum_{i:\,\text{correct}} D_t(i)}_{=1-\epsilon_t} + e^{\alpha_t}\underbrace{\sum_{i:\,\text{incorrect}} D_t(i)}_{=\epsilon_t},$$

where $\epsilon_t := \sum_{i:\,h_t(x^{(i)})\neq y^{(i)}} D_t(i)$ is the weighted error rate of h_t under distribution D_t (and D_t sums to 1, so correctly-classified weight is $1-\epsilon_t$). So

$$Z(\alpha_t) = (1-\epsilon_t)e^{-\alpha_t} + \epsilon_t e^{\alpha_t}.$$

Differentiate with respect to α_t and set to zero:

$$\frac{dZ}{d\alpha_t} = -(1-\epsilon_t)e^{-\alpha_t} + \epsilon_t e^{\alpha_t} = 0 \implies \epsilon_t e^{\alpha_t} = (1-\epsilon_t)e^{-\alpha_t} \implies e^{2\alpha_t} = \frac{1-\epsilon_t}{\epsilon_t}.$$

Taking logs and solving:

$$2\alpha_t = \ln\frac{1-\epsilon_t}{\epsilon_t} \implies \alpha_t = \frac{1}{2}\ln\frac{1-\epsilon_t}{\epsilon_t}. \qquad \blacksquare$$

The second derivative $\frac{d^2Z}{d\alpha_t^2} = (1-\epsilon_t)e^{-\alpha_t} + \epsilon_t e^{\alpha_t} = Z(\alpha_t) > 0$ (a sum of positive terms, assuming $\epsilon_t \in (0,1)$), confirming this critical point is indeed the (unique, global) minimizer. Note $\alpha_t > 0$ exactly when $\epsilon_t < 1/2$ (better than random), $\alpha_t \to \infty$ as $\epsilon_t \to 0$ (a perfect weak learner gets very high vote weight), matching the intuitive AdaBoost weighting scheme.

Exercise 6.5 — AdaBoost Training Error Bound

Prove $Z_t = 2\sqrt{\epsilon_t(1-\epsilon_t)}$ and the training error bound $\leq \exp(-2\sum_t \gamma_t^2)$.

Solution. **Deriving Z_t.** From Exercise 6.4, $Z_t = Z(\alpha_t) = (1-\epsilon_t)e^{-\alpha_t} + \epsilon_t e^{\alpha_t}$ evaluated at the optimal $\alpha_t = \frac{1}{2}\ln\frac{1-\epsilon_t}{\epsilon_t}$. Compute $e^{-\alpha_t} = \left(\frac{1-\epsilon_t}{\epsilon_t}\right)^{-1/2} = \sqrt{\frac{\epsilon_t}{1-\epsilon_t}}$ and $e^{\alpha_t} = \sqrt{\frac{1-\epsilon_t}{\epsilon_t}}$. Substituting:

$$Z_t = (1-\epsilon_t)\sqrt{\frac{\epsilon_t}{1-\epsilon_t}} + \epsilon_t\sqrt{\frac{1-\epsilon_t}{\epsilon_t}} = \sqrt{(1-\epsilon_t)^2\cdot\frac{\epsilon_t}{1-\epsilon_t}} + \sqrt{\epsilon_t^2\cdot\frac{1-\epsilon_t}{\epsilon_t}}$$

$$= \sqrt{\epsilon_t(1-\epsilon_t)} + \sqrt{\epsilon_t(1-\epsilon_t)} = 2\sqrt{\epsilon_t(1-\epsilon_t)}. \qquad \blacksquare$$

Training error bound. It is a standard AdaBoost result (following from unrolling the weight-update recursion $D_{t+1}(i) = D_t(i)e^{-\alpha_t y^{(i)}h_t(x^{(i)})}/Z_t$) that the final training error of the combined classifier $H(x) = \text{sign}(\sum_t \alpha_t h_t(x))$ is bounded by the product of the normalizers:

$$\text{Training error}(H) \leq \prod_{t=1}^T Z_t = \prod_{t=1}^T 2\sqrt{\epsilon_t(1-\epsilon_t)}.$$

(Sketch: if H misclassifies $x^{(i)}$, then $y^{(i)}\sum_t \alpha_t h_t(x^{(i)}) \le 0$, so $\exp(-y^{(i)}\sum_t \alpha_t h_t(x^{(i)})) \ge 1$; summing this indicator-dominating exponential bound over all i and tracking how the exponential weights relate to the D_{T+1} distribution recursively unrolled through the Z_t normalizers yields exactly TrainErr $\le \prod_t Z_t$.)

Define the **edge** $\gamma_t := \frac{1}{2} - \epsilon_t$ (how much better than random guessing h_t is; $\gamma_t > 0$ for a weak learner beating chance). Then $\epsilon_t = \frac{1}{2} - \gamma_t$, $1 - \epsilon_t = \frac{1}{2} + \gamma_t$, so

$$Z_t = 2\sqrt{\left(\frac{1}{2} - \gamma_t\right)\left(\frac{1}{2} + \gamma_t\right)} = 2\sqrt{\frac{1}{4} - \gamma_t^2} = \sqrt{1 - 4\gamma_t^2}.$$

Using the elementary inequality $1 - x \le e^{-x}$ (for all real x) with $x = 4\gamma_t^2$:

$$Z_t = \sqrt{1 - 4\gamma_t^2} \le \sqrt{e^{-4\gamma_t^2}} = e^{-2\gamma_t^2}.$$

Therefore

$$\text{Training error}(H) \le \prod_{t=1}^{T} Z_t \le \prod_{t=1}^{T} e^{-2\gamma_t^2} = \exp\left(-2\sum_{t=1}^{T}\gamma_t^2\right). \qquad \blacksquare$$

This shows the training error decreases *exponentially* in the number of rounds T, provided each weak learner has a (possibly small but) consistently positive edge $\gamma_t \ge \gamma > 0$ — even weak learners barely better than a coin flip drive the training error to zero exponentially fast when boosted.

Exercise 6.6 — XGBoost Leaf Weights

Derive $w_j^* = -G_j/(H_j + \lambda)$ from the Taylor-expanded objective $\sum_j [G_j w_j + \frac{1}{2}(H_j + \lambda)w_j^2] + \gamma J$.

Solution. For a fixed tree structure (fixed partition of examples into leaves $j = 1, \dots, J$), the regularized objective as a function of the leaf weights $w_1, \dots, w_J$ is

$$\text{Obj}(w) = \sum_{j=1}^{J}\left[G_j w_j + \frac{1}{2}(H_j + \lambda)w_j^2\right] + \gamma J,$$

where $G_j = \sum_{i \in \text{leaf } j} g_i$ and $H_j = \sum_{i \in \text{leaf } j} h_i$ are the summed first/second-order gradient statistics of examples falling in leaf j. Since the objective is *separable* across leaves (each w_j appears in only one summand), we can minimize each leaf's contribution independently:

$$\frac{\partial}{\partial w_j}\left[G_j w_j + \frac{1}{2}(H_j + \lambda)w_j^2\right] = G_j + (H_j + \lambda)w_j = 0 \implies w_j^* = -\frac{G_j}{H_j + \lambda}. \qquad \blacksquare$$

The second derivative is $H_j + \lambda > 0$ (assuming $\lambda \ge 0$ and $H_j > 0$, which holds for e.g. squared-error loss where $h_i = 1$ for every example, cf. Exercise 6.7), confirming this is indeed a minimum. Substituting w_j^* back gives the optimal per-leaf objective value $-\frac{1}{2}\frac{G_j^2}{H_j + \lambda}$, whose (negative) sum over leaves is exactly the "quality score" XGBoost uses to greedily choose splits.

Exercise 6.7 — XGBoost Second-Order Advantage

For $\ell(y, F) = \frac{1}{2}(y - F)^2$, compute g_i, h_i. Show $w_j^* = \bar{r}_j/(1 + \lambda/n_j)$.

Solution. g_i and h_i are the first and second derivatives of the per-example loss with respect to the current prediction F, evaluated at the current model's prediction:

$$g_i = \frac{\partial \ell(y_i, F)}{\partial F}\bigg|_{F=F^{(t-1)}(x_i)} = -(y_i - F^{(t-1)}(x_i)) = -r_i,$$

where $r_i := y_i - F^{(t-1)}(x_i)$ is the current residual for example i. And

$$h_i = \frac{\partial^2 \ell(y_i, F)}{\partial F^2} = 1$$

(the second derivative of $\frac{1}{2}(y_i - F)^2$ with respect to F is simply 1, constant).

For leaf j: $G_j = \sum_{i \in j} g_i = -\sum_{i \in j} r_i$, and $H_j = \sum_{i \in j} h_i = \sum_{i \in j} 1 = n_j$ (the number of examples in leaf j, since each contributes exactly $h_i = 1$). Substituting into the general result of Exercise 6.6:

$$w_j^* = -\frac{G_j}{H_j + \lambda} = -\frac{-\sum_{i \in j} r_i}{n_j + \lambda} = \frac{\sum_{i \in j} r_i}{n_j + \lambda}.$$

Let $\bar{r}_j := \frac{1}{n_j} \sum_{i \in j} r_i$ be the mean residual in leaf j, so $\sum_{i \in j} r_i = n_j \bar{r}_j$:

$$w_j^* = \frac{n_j \bar{r}_j}{n_j + \lambda} = \frac{\bar{r}_j}{1 + \lambda / n_j}. \qquad \blacksquare$$

Interpretation. With squared-error loss, the optimal leaf value is simply the mean residual in that leaf, *shrunk* by the factor $1/(1 + \lambda/n_j)$ — leaves with few examples (n_j small relative to λ) get shrunk more aggressively toward zero (regularizing against overfitting to a handful of points), while leaves with many examples ($n_j \gg \lambda$) are barely shrunk at all, since the shrinkage factor $\to 1$ as $n_j \to \infty$. This is directly analogous to the Ridge shrinkage pattern from Exercise 4.4, now applied per-leaf rather than per-principal-direction.

Exercise 6.8 — Stacking Data Leakage

Explain with a concrete example why training the meta-learner on same-data base predictions causes leakage. Quantify optimistic bias with a simple simulation.

Solution. **Why it's leakage.** In stacking, base models are trained on the training set, and a meta-learner is trained to combine their predictions. If the base models' predictions used as meta-features are generated by *predicting on the same data they were trained on* (i.e. each base model's in-sample/training predictions), those predictions will be unrealistically accurate — reflecting each base model's ability to *memorize* its own training data (especially for high-capacity/overfitting-prone base learners like deep trees or unregularized nets) rather than its true generalization ability. The meta-learner then learns to trust base models' (artificially good) in-sample outputs, and this trust does not transfer to new data — at inference time, base models must predict on genuinely unseen examples, where their predictions are far noisier/worse than what the meta-learner was trained to expect.

Concrete example. Suppose one base model is a deep, unpruned decision tree that perfectly memorizes (100% training accuracy) but generalizes poorly (60% test accuracy). If we generate its "meta-feature" predictions on the training set itself, that meta-feature will look like an oracle (always exactly correct) during meta-learner training — the meta-learner will learn to weight this base model very heavily. But at deployment, this same tree only achieves 60% accuracy on new data, so the

meta-learner's heavy reliance on it (learned from misleadingly perfect training-time behavior) leads to substantially degraded stacked performance relative to what was estimated during development.

Quantitative simulation sketch. Consider $n = 1000$ training points, and a base learner that is a high-capacity model with training accuracy 99% but true test accuracy 65% (a large train/test gap, i.e. overfit). If we naively use its in-sample (training-set) predictions as meta-features, the meta-learner sees a feature that is correct 99% of the time and will assign it a large, confident coefficient — but when this base model's predictions are computed on genuinely new points, they're only 65% accurate; the resulting stacked-model test accuracy will be far below what the meta-learner's training-time cross-validation loss suggested, potentially matching or even underperforming the best individual base model, and the reported (in-sample-leaked) stacking "training performance" can appear near-perfect (e.g. $\sim$98%+ meta-level training accuracy) while true held-out stacked performance might only be, say, 68–70% — an optimistic bias on the order of 25–30 percentage points in this illustrative scenario, entirely an artifact of the leakage rather than genuine ensembling benefit.

Correct procedure. Generate each base model's meta-features using out-of-fold predictions (K-fold cross-validation: for each fold, train the base model on the other $K-1$ folds and predict on the held-out fold), so that every meta-feature used to train the meta-learner reflects genuine out-of-sample performance, matching what will be seen at deployment time.

Exercise 6.9 — Gradient Boosting Pseudo-Residuals for Logistic Loss

For $\ell(y, F) = \log(1+e^{-yF})$, $y \in \{-1, +1\}$, derive $r = -\partial\ell/\partial F$. Show GB reduces to AdaBoost as step size $\to 0$.

Solution. **Pseudo-residual.** Differentiate $\ell(y, F) = \log(1 + e^{-yF})$ with respect to F:

$$\frac{\partial \ell}{\partial F} = \frac{1}{1+e^{-yF}} \cdot (-y)e^{-yF} = \frac{-y\, e^{-yF}}{1+e^{-yF}} = -y \cdot \frac{1}{1+e^{yF}} = -y\,\sigma(-yF),$$

using $\frac{e^{-yF}}{1+e^{-yF}} = \frac{1}{1+e^{yF}} = \sigma(-yF)$ (Exercise 5.1b). So the pseudo-residual (negative gradient, the target that gradient boosting fits at each stage) is

$$r = -\frac{\partial \ell}{\partial F} = y\,\sigma(-yF) = \frac{y}{1+e^{yF}}. \qquad \blacksquare$$

Interpretation: r has the same sign as y (always "pointing toward" the correct label) and its *magnitude* $|\sigma(-yF)| \in (0, 1)$ is large when yF is very negative (badly misclassified/low-confidence-correct examples) and small when yF is large and positive (confidently correct examples) — gradient boosting therefore focuses each new tree's fitting effort on the current model's poorly-classified examples, exactly analogous in spirit to AdaBoost's reweighting.

Reduction to AdaBoost as step size $\to 0$. Gradient boosting with logistic loss (this is precisely LogitBoost) builds $F_t(x) = F_{t-1}(x) + \nu\, h_t(x)$, choosing h_t to best fit the pseudo-residuals r_i via (weighted) least squares, then taking a small step of size $\nu \to 0$ (infinitesimal steps, i.e. "gradient boosting" in the strict functional-gradient-descent sense). In this infinitesimal-step-size limit, the sequence of models F_t traces out a continuous-time gradient flow on the exponential/logistic loss surface. It is a classical result (Friedman, Hastie, Tibshirani 2000, "Additive Logistic Regression") that this functional gradient descent trajectory under the exponential loss $\ell_{\exp}(y, F) = e^{-yF}$ (rather than logistic loss) reduces *exactly* to AdaBoost's reweighting scheme:

at each infinitesimal step, the weight placed on example i is proportional to the current pseudo-residual/exponential-loss-gradient magnitude $e^{-y_i F(x_i)}$, exactly matching AdaBoost's multiplicative weight update $D_{t+1}(i) \propto D_t(i) e^{-\alpha_t y_i h_t(x_i)}$ in the $\nu \to 0$ limit (each infinitesimal boosting step corresponds to one infinitesimal AdaBoost reweighting-and-refit step). For the logistic-loss variant of gradient boosting specifically (LogitBoost, as in this exercise), the same infinitesimal-step argument shows it coincides with AdaBoost's weighting dynamics to leading order near a well-classified region (where $\sigma(-yF) \approx \frac{1}{2} e^{-yF}$ for large $|yF|$, i.e. the logistic and exponential losses' gradients agree up to a constant factor when predictions are confident) — more precisely, both exponential-loss gradient boosting (exactly AdaBoost) and logistic-loss gradient boosting (LogitBoost) belong to the same family of "AnyBoost"-style functional gradient descent algorithms differing only in the choice of convex margin-based loss, and in the vanishing-step-size limit both perform (loss-specific) coordinate/functional gradient descent in the direction of the current pseudo-residual, which is the shared structural reason AdaBoost is recovered as the special case with exponential loss.